Linux/Unix/OpenGL C++开发题目与核心知识点全览

创建人：superrrr 创建时间：2026.03.29 15:12

一、Linux环境下C++编程题目

1. 进程管理

题目：防止僵尸进程的代码示例
答案：

```
 #include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handler(int sig) {
    int status;
    if (waitpid(-1, &status, WNOHANG) >= 0) {
        printf("child is die\n");
    }
}

int main() {
    signal(SIGCHLD, handler);
    pid_t pid = fork();

    if (pid > 0) { // 父进程循环等待
        while (1) {
            sleep(2);
        }
    } else if (pid == 0) { // 子进程
        printf("i am child, i die\n");
        exit(0);
    }

    return 0;
}
```

题目：进程间通信（管道）实现示例
答案：

```
  #include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INPUT  0 // 读文件描述符
#define OUTPUT 1 // 写文件描述符

int main() {
    int fd[2];
    pipe(fd); // 创建管道

    pid_t pid = fork(); // 创建子进程

    if (pid < 0) {
        printf("fork error!\n");
        exit(-1);
    } else if (pid == 0) { // 子进程
        printf("Child process is starting...\n");
        close(fd[INPUT]); // 关闭读端
        write(fd[OUTPUT], "hello douya!", strlen("hello douya
        exit(0);
    } else { // 父进程
        printf("Parent process is starting......\n");
        close(fd[OUTPUT]); // 关闭写端
        char buf[255];
        int output = read(fd[INPUT], buf, sizeof(buf));
        printf("%d bytes of data from child process: %s\n", o
    }

    return 0;
}
```

额外知识点汇总：

- **僵尸进程产生原因：**子进程退出后，父进程未调用 `wait()` 或 `waitpid()` 回收其资源。
- **避免僵尸进程：**在父进程中捕获 SIGCHLD 信号并调用 `waitpid()`，或使用 `fork()` 两次。
- **进程间通信方式：**管道、信号、共享内存、消息队列、信号量、套接字等。
- **进程创建方式：**系统初始化、`fork()`创建子进程、用户交互请求、批处理作业初始化。
- **进程调度算法：**先来先服务(FCFS)、短进程优先(SPF)、时间片轮转(RR)、多级反馈队列等。
- **多进程vs多线程：**

- **多进程**适用于安全性、稳定性要求高的场景。
- **多线程**适用于频繁通信、切换、创建和销毁的场景。

2. 系统调用与底层操作

题目：安全的寄存器位反转函数
编写一个安全的函数将地址为 `0x1F000010`的32位寄存器REGn的指定位 `nbit` 反转。
答案：

```
  void bit_reverse(uint32_t nbit)
{
    *((volatile unsigned int *)0x1F000010) ^= (0x01 << nbit);
}
```

题目：用户进程调用 `read()`/`write()` API时，内核做了什么？
答案：用户进程调用 `read()`/`write()` 后进入内核空间 → 调用内核的 `sys_read()`/`sys_write()` 函数 → 内核函数进一步调用 `fop.read()`/`fop.write()` 指向的函数，也就是驱动程序里具体的读写函数。

题目：系统调用的作用？
答案：

1. 为应用程序提供访问硬件资源的统一接口，以至于应用程序不必关心具体的硬件操作细节。
2. 对系统内核进行保护，保证系统的稳定和安全，因为系统调用规定了用户进程进入内核的具体方式以及所能访问的数据范围。

题目：实现 `my_strcpy()` 函数
答案（来自文档内容）：

```
  #include <stdio.h>
char* strcpy(char *strDest, const char* strSrc) {
    char *p = strDest;
    if(strDest == NULL || strSrc == NULL) {
        return NULL;
    }
    while((*strDest++ = *strSrc++) != '\0');
    return p;
}
```

题目：检查输入参数是否为 2 的幂次方
答案：

```
  bool check(unsigned int d) {
    return (d & (d - 1)) == 0;
}
```

题目：V4L2摄像头应用编写框架
答案：

1. **打开摄像头设备**：使用 `open` 系统调用打开摄像头设备文件（例如 `/dev/video0`）。
2. **获取并设置摄像头设备参数**：使用 `ioctl` 系统调用，执行 `VIDIOC_QUERYCAP`命令获取设备信息；执行 `VIDIOC_S_FMT`命令设置视频帧格式、分辨率等参数。
3. **启动摄像头设备**：使用 `ioctl` 系统调用，执行 `VIDIOC_STREAMON` 命令启动摄像头设备。
4. **读取视频帧数据**：使用 `read` 系统调用从摄像头设备读取视频帧数据。
5. **处理视频帧数据**：对读取到的视频帧数据进行图像处理、视频编码、储存等。
6. **关闭摄像头设备**：使用 `ioctl` 执行 `VIDIOC_STREAMOFF` 命令关闭设备，并使用 `close` 关闭设备文件。

题目：内核程序与应用程序申请内存有什么区别?
答案：内核中申请内存空间用的是函数 `kmalloc()`、`kzalloc()`、`vmalloc()`，应用程序申请内存用的函数是 `malloc()`。

题目：用户空间与内核空间通信方式有哪些?
答案：

1. **系统调用**：用户空间进程通过系统调用进入内核空间，访问指定的内核空间数据。
2. **驱动程序**：用户空间进程可以使用封装后的系统调用接口访问驱动设备节点，以和运行在内核空间的驱动程序通信。
3. **共享内存mmap**：在代码中调用接口，实现内核空间与用户空间的地址映射。
4. **copy_to_user() / copy_from_user()**：在驱动程序中调用接口，实现用户空间与内核空间的数据拷贝操作。

3. 文件操作

题目：打开一个文件，并读取从第 100 字节开始的 50 字节数据。
答案：

```
 #include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    char buffer[50];
    fp = fopen("test.txt", "rb");
    if (fp == NULL) {
        perror("File opening failed");
```

```
        return EXIT_FAILURE;
    }
    fseek(fp, 100, SEEK_SET); // 移动到第100字节
    fread(buffer, 1, 50, fp); // 读取50字节
    buffer[50] = '\0'; // 假设是字符串，添加结束符
    printf("%s\n", buffer);
    fclose(fp);
    return 0;
}
```

题目：文件中有一组整数，要求排序后输出到另一个文件中。

答案（C++示例）：

```
 #include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>

int main() {
    std::ifstream inFile("input.txt");
    std::ofstream outFile("output.txt");
    std::vector<int> numbers;
    int num;

    // 读取文件中的整数
    while (inFile >> num) {
        numbers.push_back(num);
    }

    // 排序（使用标准库排序算法）
    std::sort(numbers.begin(), numbers.end());

    // 输出到另一个文件
    for (int n : numbers) {
        outFile << n << " ";
    }

    inFile.close();
    outFile.close();
    return 0;
}
```

题目：统计input字符串中各单词的出现频率，由高到低排序输出为output（对于频率相同的，按原顺序不动）。

代码框架：

```
 char *input = "this is life that is hope";
char output[] = "is this life that hope";
void sort_string(char *input, char *output)
{
```

```
//请实现
//提示：从input字符串里，解析单词，存入链表，链表结点应包含单词出现次数这
}
```

解题思路摘要：

1. 遍历`input`字符串，提取单词存入链表并与计数器关联。
2. 使用稳定的排序算法（冒泡排序、插入排序），根据出现频率排序，当频率相等时保留原始顺序（稳定）。
3. 拼接至`output`数组。

题目：为如下项目结构编写Makefile：

```
 helloworld
|- file2.h
|- file1.cpp
|- file2.cpp
```

答案：

```
 CC = g++
CFLAGS = -Wall -g

all: helloworld

helloworld: file1.o file2.o
    $(CC) $(CFLAGS) -o helloworld file1.o file2.o

file1.o: file1.cpp file2.h
    $(CC) $(CFLAGS) -c file1.cpp

file2.o: file2.cpp file2.h
    $(CC) $(CFLAGS) -c file2.cpp

clean:
    rm -f *.o helloworld
```

题目：实现字符串数字相加的函数（字符串表示的数字可能过长，不能用基本数据类型表示）。

解题思路摘要：

1. 模仿手算加法，必须从末尾开始加法。
2. 注意处理进位，进位 = 求和/10 ，当前位置的结果 = (和 %10)。
3. 最后的结果字符串需要一个反转。

4. 网络编程

概念题：

- **题目**：请你说一说**IP地址**作用，以及**MAC地址**作用？
  - **答案**：**IP地址**为网络和主机提供逻辑地址，用于跨网络寻址；**MAC地址**是硬件地址，用于局域网内直接寻址。
- **题目**：TCP、UDP的区别？
  - **答案**：TCP面向连接、可靠、有序、有拥塞控制、速度慢；UDP无连接、不可靠、无序、无拥塞控制、速度快。
- **题目**：HTTP协议会话结束标志怎么截出来？
  - **答案**：看TCP连接是否有断开的四次挥手阶段。
- **题目**：套接字socket有哪几类型？
  - **答案**：1. `SOCK_STREAM`（TCP）； 2. `SOCK_DGRAM`（UDP）； 3. `SOCK_RAW`（原始套接字）。
- **题目**： Socket网络编程中，哪五个元素可以明确标识一条连接？
  - **答案**： **五元组**: 协议、源IP地址、源端口号、目的IP地址、目的端口号。

流程函数题（基于TCP）：
题目：socket编程中服务器端和客户端主要用到哪些函数？
答案（服务器流程）：`socket()` -\u003e `bind()` -\u003e `listen()` -\u003e `accept()` -\u003e `send()/recv()` -\u003e `close()`。
答案（客户端流程）：`socket()` -\u003e `connect()` -\u003e `send()/recv()` -\u003e `close()`。

命令题：

- **题目**：Linux中，如何查看客户端与服务端是否成功建立了TCP连接？
  - **答案**：`netstat -ant | grep “10.157.123.100:8600” | grep “10.157.123.200:8700” | grep “ESTABLISHED”`。
- **题目**：如何测试DNS设置是否正确？
  - **答案**：使用`ping`测试连通性；使用`nslookup`或`dig`查询DNS解析；使用`ipconfig`（Windows）或`ifconfig`（Linux）查看配置。

5. 多线程

概念题：

- **题目**：进程与线程的区别？
  - **答案**：进程是资源分配的基本单位，拥有独立地址空间；线程是CPU调度的基本单位，共享进程资源。

- **题目**：何时使用多进程 vs 多线程？
  - **答案**：多进程用于安全性、稳定性要求高或需扩展到多机的场景；多线程用于频繁创建/销毁、需要大量数据共享、或利用多核提高性能的场景。
- **题目**：多线程在什么情况下效率更高？单线程呢？
  - **答案**：多线程在并发且无需频繁访问共享数据时效率高。若线程切换过于频繁或共享数据竞争激烈，单线程可能更好。
- **题目**：如何减少锁的使用以提高并发效率？
  - **答案**：减少锁的粒度和持有时间；使用无锁数据结构；在读多写少场景使用读写锁；使用线程本地存储(TLS)。
- **题目**：什么是死锁？如何避免？
  - **答案**：死锁指多个线程因互相等待资源而无法推进。避免方法：按固定顺序加锁；加锁设置超时；避免在持有一个锁时申请另一个；一次性申请所有需要的锁。
- **题目**：用户级线程和内核级线程的区别？Linux下是哪种？
  - **答案**：用户级线程由用户库管理，开销小但无法利用多核；内核级线程由OS管理，开销大但能利用多核。Linux支持内核级线程（通过轻量级进程LWP实现）。

同步机制题：

1. **信号量 (Semaphore)**：控制同时访问共享资源的线程数量。`sem_init`，`sem_wait`，`sem_post`。
2. **互斥锁 (Mutex)**：保证同一时刻只有一个线程进入临界区。`pthread_mutex_init`，`pthread_mutex_lock`，`pthread_mutex_unlock`。
3. **条件变量 (Condition Variable)**：用于线程等待某个条件成立，常与互斥锁配合。`pthread_cond_wait`，`pthread_cond_signal`，`pthread_cond_broadcast`。
4. **读写锁 (Read-Write Lock)**：允许多个线程同时读，但只允许一个线程写。
5. **自旋锁 (Spinlock)**：忙等待锁，适用于锁持有时间极短的场景。

核心函数题：

操作	函数
创建线程	`pthread_create(\u0026tid, NULL, thread_func, arg)`

操作	函数
线程加入/回收	`pthread_join(tid, NULL)`
线程分离	`pthread_detach(tid)`
条件变量等待	`pthread_cond_wait(\u0026cond, \u0026mutex)`
条件变量唤醒	`pthread_cond_signal(\u0026cond)` 或 `pthread_cond_broadcast(\u0026cond)`
信号量操作	`sem_wait(\u0026sem)` (P操作)， `sem_post(\u0026sem)` (V操作)

线程池题：

- **题目：**线程池有什么好处？线程数量如何确定？
  - **答案：**线程池避免了频繁创建/销毁的开销。
  - **线程数量公式：**
    - 最佳线程数 = (线程等待时间/线程CPU计算时间 + 1) × CPU核心数（精确做法）。
    - CPU密集型：线程数 ≈ CPU核心数 + 1。
    - I/O密集型：线程数 ≈ 2 × CPU核心数 + 1。

二、Unix内核环境下C++编程题目

在Unix内核环境，特别是嵌入式Linux开发中，除了掌握通用的用户空间C++编程知识外，还必须深入理解如何在内核空间进行编程，以及与硬件、系统核心进行交互。本部分题目聚焦于内核模块、驱动开发及底层系统交互等核心领域。

📌 1. 系统调用实现

系统调用是用户程序访问内核服务与硬件资源的统一且安全的接口。

- **题目1：请简述系统调用的核心作用？**

- **答案**：1) 为上层应用程序提供访问底层硬件资源的统一接口，使得应用程序无需关心具体的硬件操作细节。2) 对系统内核进行保护，通过规定用户进程进入内核的具体方式及其可访问的数据范围，保证了系统的稳定性和安全性。

- **题目2：描述用户进程调用 `read()`/`write()` API时，内核的具体工作流程？**
  - **答案**：用户进程调用 `read()`/`write()` → 通过软中断或专用指令陷入内核态 → 执行内核对应的 `sys_read()`/`sys_write()` 系统调用函数 → 内核函数进一步调用 `file_operations` 结构体中对应的 `fop.read()`/`fop.write()` 函数指针，最终由设备驱动完成实际的硬件读写操作。

- **题目3：用户空间与内核空间可以通过哪些方式进行通信？**
  - **答案**：常见方式包括：1) **系统调用**，是标准接口。2) **驱动设备节点**，用户程序通过 `open`、`read`、`write`、`ioctl` 等操作设备文件与驱动交互。3) **共享内存(mmap)**，将内核空间或物理内存映射到用户进程地址空间。4) 在驱动中利用 `copy_to_user()`/`copy_from_user()` 函数复制数据。5) **Procfs (`/proc`)** 或 **Sysfs (`/sys`)** 虚拟文件系统。6) **Netlink套接字**，一种用于内核与用户态进程通信的IPC机制。

- **题目4：编写一个安全的函数，将地址为 `0x1F000010` 的32位寄存器 `REGn` 的第 `nbit` 位反转。**
  - **答案**：

    ```
    void bit_reverse(uint32_t nbit) {
      *((volatile unsigned int *)0x1F000010) ^= (0x01 << nb
    }
    ```

    注：使用 `volatile` 关键字防止编译器优化对寄存器地址的访问；使用异或 (`^`) 操作实现指定位反转。

🧠 2. 内核内存管理

内核态下的内存管理与用户态有显著区别，涉及物理内存的直接操作。

- **题目1：在内核程序中申请内存使用什么函数？与用户空间的 `malloc` 有何区别？**
  - **答案**：
    - **内核函数**：`kmalloc()`, `kzalloc()`, `vmalloc()`。
    - **区别**：
      - `kmalloc`/`kzalloc`：分配**物理地址连续**的内存（虚拟地址也连续），大小有限制（通常≤128KB），性能高，适用于DMA等场景。`kzalloc` 会初始化为0。

    - `vmalloc`：分配**虚拟地址连续但物理地址不一定连续**的内存，大小无严格限制，但访问效率相对较低，不能在中断上下文中调用。
    - `malloc` (用户空间)：通过系统调用分配**虚拟地址连续**的内存，物理地址由内核管理，一般对应用程序透明。
- **题目2：为什么在驱动中操作硬件的物理绝对地址前，通常需要使用 `ioremap()`？**
  - **答案**：在现代操作系统（如Linux）的虚拟内存管理下，CPU指令访问的都是虚拟地址。对于直接映射到CPU地址空间的硬件寄存器（物理地址），内核不能直接访问。`ioremap()` 的作用就是将一段**物理地址空间**映射到内核的**虚拟地址空间**，使得驱动程序可以通过返回的虚拟地址指针来安全地读写这些硬件寄存器（使用 `readl`/`writel` 等接口）。

⚙️ 3. 内核模块与驱动开发

内核模块允许动态扩展内核功能，设备驱动则是内核与硬件交互的桥梁。

- **题目1：Linux中引入模块机制有什么好处？**
  - **答案**：1) 模块可以在系统运行时动态加载和卸载，无需重新编译和重启整个内核，极大增强了系统的灵活性。2) 模块在退出时能通过 `module_exit` 函数集中释放其占用的所有资源，有助于缩短开发和调试周期，也使得内核功能可以模块化开发。
- **题目2：`insmod` 和 `modprobe` 都可以加载内核模块，描述两者的差异。**
  - **答案**：`modprobe` 可以自动解决模块之间的**依赖关系**（通过读取 `/lib/modules/#uname -r/modules.dep` 文件），并按需加载依赖模块。而 `insmod` 仅仅将指定的 `.ko` 文件插入内核，不处理任何依赖。如果模块没有依赖，两者皆可；若存在依赖，必须使用 `modprobe`。
- **题目3：编写一个基本的字符设备驱动框架，需要实现哪些核心组成部分？**
  - **答案核心结构**：
    1. **定义文件操作结构体 `struct file_operations`**：填充 `.owner`, `.open`, `.release`, `.read`, `.write`, `.unlocked_ioctl` 等成员函数指针。
    2. **模块初始化函数 (`module_init`)**：
       - 调用 `alloc_chrdev_region` 动态申请设备号。
       - 使用 `cdev_init` 初始化 `struct cdev`，并将其与 `file_operations` 绑定。
       - 调用 `cdev_add` 将字符设备添加到内核。
       - (可选) 使用 `device_create` 和 `class_create` 在 `/dev/` 自动创建设备节点。

3. **模块退出函数 (`module_exit`)**：与初始化过程相反，进行资源释放、设备删除、设备号注销等操作。

- **题目4：自旋锁 (spinlock) 和信号量 (semaphore) 在互斥使用时有何重要区别？在中断服务程序 (ISR) 中应使用哪个？**
  - **答案**：
    - **区别**：使用**自旋锁**的进程/线程在获取锁失败时，会**忙等待**（循环检查），不会睡眠，适用于临界区极短的场景。使用**信号量**的进程/线程在获取信号量失败时，会进入**睡眠状态**，让出CPU，适用于临界区可能较长的场景。
    - **ISR中使用**：必须使用**自旋锁**。因为中断上下文（尤其是顶半部）不允许睡眠或调度，一旦睡眠就无法被唤醒。

⚠️ 4. 中断处理

中断处理是内核响应硬件事件的关键机制，编写中断服务程序有严格的限制。

- **题目1：中断处理为何通常分为“上半部”(top half)和“下半部”(bottom half)？它们各自的任务是什么？**
  - **答案**：为了尽可能缩短硬件中断被屏蔽的时间，提高系统响应能力。
    - **上半部**：即中断服务程序 (ISR)，**快速响应**硬件中断，执行**紧急、不可延迟**的操作，如读取硬件状态、清除中断标志。要求**执行时间非常短**，且**不允许阻塞**。
    - **下半部**：用于处理**可以延迟执行**、相对耗时的工作。Linux提供了多种下半部机制（如软中断、tasklet、工作队列等），将上半部中标记的任务推迟到更安全、允许调度的上下文中执行。
- **题目2：找出以下中断服务程序代码中的问题并说明原因。**

```
__interrupt double compute_area(double radius) {
    double area = PI * radius * radius;
    printf(" Area = %f", area);
    return area;
}
```

  - **答案**：该ISR存在**多个严重问题**：
    1. **不能有返回值**：ISR由硬件触发，并非普通函数调用，没有调用者接收其返回值。
    2. **不能有参数**：ISR不是被程序代码调用，无法以参数形式传递 `radius`。

3. **执行浮点运算 (`double`)**：在许多处理器架构和编译器环境中，浮点运算单元（FPU）的状态可能不是可重入的，在中断上下文中直接使用可能导致内核状态混乱。
4. **调用 `printf`**：`printf` 是标准库函数，它内部可能使用全局缓冲区和锁，在不可调度的中断上下文中使用可能导致**死锁**、**重入问题**，且其执行速度慢，违背ISR“快进快出”原则。

- **题目3：在中断服务程序中，有哪些必须遵守的编程准则？**
  - **答案核心准则**：
    1. **执行时间尽可能短**，避免长时间占用中断线。
    2. **禁止睡眠或阻塞**，中断上下文不参与进程调度。
    3. **避免使用不可重入的函数**，如标准I/O库函数 (`printf`, `malloc` 等)。
    4. **谨慎处理共享数据**，必须使用自旋锁等适合中断上下文的同步机制。
    5. **通常避免进行浮点运算**（除非明确保存和恢复了FPU上下文）。

三、OpenGL/WinGUI图形编程C++题目

> **注**：根据现有资料，以下主要整理归纳了Win32 API与MFC的相关编程题目。DirectX详细题目（如Direct3D初始化）及图形引擎内部管线题目（如光栅化、深度测试）的详细题目与答案未在资料中提供。

一、Win32 API编程题目

1. 窗口与消息机制

- **题目1**：Windows程序的入口是哪里？
  - **答案**：`WinMain`函数。
- **题目2**：创建一个窗口的步骤是什么？
  - **答案**：
    1. 填充一个窗口类结构；
    2. 注册这个窗口类；
    3. 创建窗口；
    4. 显示窗口；
    5. 更新窗口。

- **题目3**：Windows消息系统由哪几部分构成？
  - **答案**：
    1. **消息队列**：操作系统为进程维护的消息队列，程序从中获取并处理消息。
    2. **消息循环**：应用程序不断从队列获取、处理消息的循环。
    3. **消息处理**：消息循环将消息派发到相关窗口，由窗口过程函数处理。
- **题目4**：Windows消息分为几类？并对各类做简单描述。
  - **答案**：
    1. **窗口消息**：与窗口相关的消息，除`WM_COMMAND`外所有以`WM_`开头的消息。
    2. **命令消息**：处理用户请求，以`WM_COMMAND`表示。
    3. **控件通知消息**：统一由`WM_NOTIFY`表示。
    4. **用户自定义消息**。
- **题目5**：如何自定义消息？
  - **答案**：使用`WM_USER`和`WM_APP`两个宏来自定义消息。
- **题目6**：什么是消息映射？
  - **答案**：消息映射是让程序员指定MFC类处理某个消息，并编写处理函数以实现消息处理功能。
- **题目7**：`SendMessage`和`PostMessage`有什么区别？
  - **答案**：
  - `SendMessage`是阻塞的，必须等待消息被处理后才继续执行下一行代码。
  - `PostMessage`是非阻塞的，无论消息是否处理，都立即执行下一行代码。
- **题目8**：Windows消息调度机制是什么？
  - **答案**：消息队列。
- **题目9**：构成Win32 API函数的三个动态链接库是什么？
  - **答案**：内核库、用户界面管理库、图形设备界面库。
- **题目10**：如何定义和实现一个类的成员函数为回调函数？
  - **答案**：在成员函数前加`CALLBACK`宏定义，实现与普通成员函数相同。

2. 对话框与控件编程

- **题目11**：模态对话框和非模态对话框有什么区别？
  - **答案**：

1. **调用规则不同**：模态对话框用`DoModal()`调用；非模态对话框通过属性和`ShowWindow()`显示。
2. **交互限制**：模态对话框关闭前用户不能进行其他操作；非模态对话框可以。
3. **创建要求**：非模态对话框必须编写自己的公有构造函数并调用`Create()`函数。

- **题目12**：从EDIT框中取出数据给关联的变量，或把变量的数据显示在EDIT框上的函数是什么？
  - **答案**：`UpdateData(TRUE)`从EDIT框取数据到变量；`UpdateData(FALSE)`将变量数据显示到EDIT框。

3. GDI绘图

- **题目13**：简单介绍GDI（图形设备接口）。
  - **答案**：GDI是Windows中执行与设备无关图形输出的函数库，用于在不同输出设备上产生图形和文字。

4. 内存与数据类型

- **题目14**：在Win32下，`char`、`int`、`float`、`double`各占多少位？
  - **答案**：`char`（8位），`int`（32位），`float`（32位），`double`（64位）。
- **题目15**：`CMemoryState`主要功能是什么？
  - **答案**：查看内存使用情况，解决内存泄露问题。
- **题目16** (代码题)：以下为Windows NT下的32位C++程序，请计算`sizeof`的值。

```
 char str[] = “Hello”;
char *p = str;
int n = 10;
void Func(char str[100]); // 计算函数体内 sizeof(str)
void *p = malloc(100); // 计算 sizeof(p)
```

  - **答案**：

    `sizeof(str) = 6`

    `sizeof(p) = 4`

    `sizeof(n) = 4`

    `sizeof(str) = 4` (参数退化为指针)

`sizeof(p) = 4` (指针大小)

5. 网络通信

- **题目17**：简述Winsock建立连接的主要实现步骤。
  - **答案**：
  - **服务器端**：`socket()` → `bind()` → `listen()` → `accept()` → `send()/recv()` → `closesocket()`
  - **客户端**：`socket()` → `connect()` → `send()/recv()` → `closesocket()`

二、MFC框架编程题目

- **题目1**：简述Visual C++、Win32 API和MFC之间的关系。
  - **答案**：

  1. Visual C++是以C++为基础的集成开发环境。
  2. Win32 API是32位Windows以C/C++形式提供的应用程序接口。
  3. MFC是对Win32 API的封装，简化了开发过程。

三、OpenGL基础编程题目

> **注**：以下题目为基于C++和计算机图形学基础知识编写的典型示例，供学习参考。**依据现有<搜集资料>，OpenGL、着色器、DirectX等图形技术相关的具体题目与答案未直接提供。**

- **题目1** (基础绘制)：使用现代OpenGL（可编程管线）和C++，简述绘制一个彩色三角形的主要步骤，涉及顶点数据、着色器程序和VAO/VBO。
  - **答案要点**：

  1. 定义包含位置和颜色的顶点数据数组。
  2. 创建并绑定VBO，将数据传入GPU；创建并绑定VAO，使用`glVertexAttribPointer`设置顶点属性。
  3. 编写、编译并链接着色器（顶点着色器与片段着色器）。
  4. 在渲染循环中：激活着色器程序、绑定VAO、调用`glDrawArrays(GL_TRIANGLES, 0, 3)`进行绘制。

- **题目2** (坐标变换)：在OpenGL中，一个顶点从局部坐标变换到屏幕坐标需要经过哪些空间？写出主要的变换矩阵及其作用。

- **答案要点**：

1. **模型矩阵**：负责物体从局部空间到世界空间的平移、旋转、缩放。
2. **视图矩阵**：负责从世界空间到观察空间的变换，模拟相机位置和朝向。
3. **投影矩阵**：负责从观察空间到裁剪空间的变换，定义视野、宽高比、近远平面，将3D坐标投影到标准设备坐标系。

四、三种环境通用C++核心知识题目

无论开发环境是Linux用户空间、Unix内核模块，还是Windows/OpenGL图形应用，以下C++语言本身的核心概念与机制都是必须掌握的基础。本章题目聚焦于这些跨平台的通用知识。

模板编程

1. **题目**：函数模板与类模板有什么区别？
   答案：函数模板的实例化是由编译程序在处理函数调用时自动完成的；而类模板的实例化必须由程序员在程序中显式地指定。
2. **题目**：写出下面函数的功能（模板函数）

```
  template <class Type>
void WE(Type a[], Type b[], int n) {
   for (int i = 0; i < n; i++)
       b[n-i-1] = a[i];
}
```

   答案：该模板函数将数组 `a` 的每个元素按逆序放入数组 `b` 中。
3. **题目**：定义一个类模板与定义一个函数模板的格式相同，必须以关键字_____开始。
   答案：`template`
4. **题目**：C++中什么数据分配在栈或堆中？
   答案：
   - **栈**：存放局部变量、函数调用参数、函数返回值、函数返回地址。由系统管理。
   - **堆**：程序运行时动态申请，`new` 和 `malloc` 申请的内存就在堆上。
5. **题目**：在不用第三方参数的情况下，交换两个整型参数的值（可使用模板实现通用交换）。

答案：一种使用模板的通用交换方法（基于异或）：

```
  template<typename T>
void swap_xor(T& a, T& b) {
    a ^= b;
    b ^= a;
    a ^= b;
}
// 注：此方法仅适用于整型等支持异或运算的类型。
```

异常处理

1. **题目**：简述C++异常处理的执行流程（涉及 `throw` 和 `try-catch`）。
   答案：使用 `throw` 抛出异常对象（可以是基本类型或类类型）。使用 `try-catch` 块捕获并处理异常。**执行流程**：当抛出异常后，程序会沿着函数调用的逆序依次析构局部对象，直到遇到 `try` 块的右花括号，然后根据异常类型匹配对应的 `catch` 块进行处理。
2. **题目**：什么是异常规格（Exception Specification）？现代C++推荐用什么替代？
   答案：异常规格在函数形参表后声明，形式为 `throw(异常类型1, 异常类型2, ...)`。`throw()` 表示函数不会抛出任何可捕获的异常。现代C++中已较少使用异常规格，因为容易引发意外行为；建议使用 `noexcept` 替代。
3. **题目**：为什么永远不要在析构函数中抛出异常？
   答案：应在析构函数内使用 `try-catch` 拦截所有可能异常，防止异常传播导致程序终止。若析构函数抛出异常且未在内部捕获，可能引发程序直接 `std::terminate`。
4. **题目**：如何保证构造函数中的异常安全？
   答案：若构造函数抛出异常，对象将成为“不完整对象”，其析构函数不会被自动调用。因此，必须在抛出异常前手动释放已申请的资源（如动态内存），或使用“资源获取即初始化”（RAII）技术，避免资源泄漏。

STL（标准模板库）

1. **题目**：`map`和`set`有什么区别？分别是怎么实现的？
   答案：
   - **相同点**：都是C++的关联容器，底层实现都是**红黑树**（RB-Tree）。

- **区别**：`map`存储键值对（key-value），`set`只存储关键字（key）。`set`的迭代器是`const`的，不允许修改元素值；`map`允许修改`value`，但不允许修改`key`。`map`支持下标操作（`[]`），`set`不支持。

2. **题目**：`vector`和`list`的区别是什么？
   答案：
   - **底层结构**：`vector`底层是**数组**；`list`是**双向链表**。
   - **内存**：`vector`是**连续内存**；`list`是**非连续内存**。
   - **访问性能**：`vector`支持**随机访问**（O(1)）；`list`不支持随机访问（O(n)）。
   - **插入/删除**：`vector`在中间插入/删除会导致**内存拷贝**（O(n)）；`list`插入/删除只需**调整指针**（O(1)）。
3. **题目**：STL中`map`与`unordered_map`有什么区别？
   答案：
   - **底层结构**：`map`使用**红黑树**；`unordered_map`使用**哈希表**。
   - **有序性**：`map`遍历结果**有序**（按key排序）；`unordered_map`遍历结果**无序**。
   - **性能**：`unordered_map`查找、插入、删除平均O(1)，但最坏情况O(n)；`map`始终稳定在O(log n)。
4. **题目**：STL中迭代器有什么作用？有指针为何还要迭代器？
   答案：迭代器提供一种**统一的方式**顺序访问容器中的元素，而不暴露容器内部结构。迭代器是类模板，模拟指针行为，但能适应不同容器（如链表、数组）的遍历逻辑，抽象不同容器的访问逻辑，实现通用遍历。
5. **题目**：`resize`和`reserve`的区别？
   答案：
   - **resize**：改变容器中元素的**数量**（`size`），可能会添加或删除元素。
   - **reserve**：改变容器的**容量**（`capacity`），只分配内存，不创建元素。

面向对象编程

1. **题目**：面向对象的三个基本特征是什么？
   答案：**封装、继承、多态**。
2. **题目**：构造函数和析构函数是否可以被重载？为什么？
   答案：构造函数可以重载，析构函数不可以。因为构造函数可以有多个且可以带参数，而析构函数只能有一个且不能带参数。
3. **题目**：什么是虚函数？它如何实现多态？
   答案：虚函数是在基类中用`virtual`声明的成员函数，允许在派生类中被重新定

义（覆盖）。通过基类指针或引用调用虚函数时，根据实际对象类型动态绑定到派生类函数，实现**运行时多态**。

4. **题目**：重载（overload）和重写（override，也称覆盖）的区别？
   答案：
   - **重载**：在**同一作用域内**（如一个类中），函数名相同但参数列表不同，**编译时**确定。
   - **重写**：在**派生类**中重新定义基类的**虚函数**，函数签名必须相同，**运行时**通过虚函数表动态绑定。
5. **题目**：深拷贝与浅拷贝的区别？
   答案：
   - **浅拷贝**：仅复制指针，拷贝后多个指针指向同一内存空间。
   - **深拷贝**：复制指针及指向的内容，拷贝后指针指向不同内存空间。深拷贝避免多次释放同一内存导致错误。
6. **题目**：`struct` 和 `class` 在 C++ 中的默认区别是什么？
   答案：`struct` 的成员默认访问权限是 **`public`**，`class` 的成员默认访问权限是 **`private`**。

内存管理

1. **题目**：简述 `malloc/free` 和 `new/delete` 的主要区别。
   答案：

特性	`malloc/free`	`new/delete`
语言	C/C++库函数	C++运算符
返回类型	`void*`，需强转	对象类型指针
计算大小	需显式指定字节数	自动计算
构造/析构	不调用	自动调用构造函数/析构函数
重载	不可重载	`new/delete`可被重载

特性	`malloc/free`	`new/delete`
失败	返回`NULL`	抛出`bad_alloc`异常

2. **题目**：什么是内存泄漏？如何避免？
   答案：内存泄漏指程序中已动态分配的堆内存，由于某种原因**未能释放或无法释放**。避免方法：配对使用 `new/delete` 或 `malloc/free`；使用智能指针（如 `std::unique_ptr`，`std::shared_ptr`）；释放后指针置 `NULL`；使用内存检测工具。
3. **题目**：分析以下代码是否存在问题：

```
void Test(void) {
    char *str = (char *) malloc(100);
    strcpy(str, \"hello\");
    free(str);
    if(str != NULL) {
        strcpy(str, \"world\");
        printf(str);
    }
}
```

   答案：存在问题。`free(str)` 后，`str` 成为**野指针**，`if(str != NULL)` 判断无效（因为 `free` 不会自动置指针为 `NULL`），后续的 `strcpy` 操作篡改已释放内存，行为未定义。
4. **题目**：什么是内存对齐？为什么需要内存对齐？
   答案：内存对齐指各种类型数据按照一定规则（通常是其自身大小的整数倍地址）在空间上排列。原因：**性能**（CPU访问对齐内存更快）；**平台硬件要求**（某些架构只能访问对齐地址，否则引发异常）。

智能指针

1. **题目**：什么是智能指针？它的主要作用是什么？
   答案：智能指针是一个**类**，用于存储指向动态分配对象的指针。主要作用是通过**自动管理资源生命周期**（如引用计数、作用域结束自动释放），帮助避免内存泄漏、二次释放等问题。
2. **题目**：`std::shared_ptr` 如何管理资源？它可能带来什么问题？如何解决？
   答案：`std::shared_ptr` 通过**引用计数**管理资源，计数归零时自动释放。可能带来

循环引用问题（两个或多个 `shared_ptr` 互相引用，计数永不为零，导致内存泄漏）。解决方法是使用 `std::weak_ptr`（弱引用指针）打破循环。

3. **题目**：`std::unique_ptr` 和 `std::shared_ptr` 的主要设计区别是什么？
   答案：`std::unique_ptr` 体现**独占所有权**，同一时刻只能有一个 `unique_ptr` 指向一个对象，所有权可通过 `std::move` 转移。`std::shared_ptr` 体现**共享所有权**，多个 `shared_ptr` 可以共享同一对象。

4. **题目**：`std::weak_ptr` 如何安全地访问所观察的资源？
   答案：通过 `lock()` 方法，返回一个 `std::shared_ptr`。若原对象未被释放，则返回有效的 `shared_ptr`；否则返回空的 `shared_ptr`。例如：

```
if (auto sp = weakPtr.lock()) {
    // 安全使用 sp
}
```

多线程编程

1. **题目**：线程和进程的主要区别是什么？
   答案：
   - **资源**：进程是**资源分配**的基本单位（拥有独立地址空间）；线程是**CPU调度**的基本单位，**共享**进程资源。
   - **开销**：创建、销毁、切换进程的开销**远大于**线程。
   - **健壮性**：一个线程崩溃可能导致整个进程崩溃；进程间相互独立。

2. **题目**：请列举至少三种线程同步的方式。
   答案：**互斥锁 (Mutex)**、**信号量 (Semaphore)**、**条件变量 (Condition Variable)**、**读写锁**。

3. **题目**：互斥锁 (Mutex) 和 信号量 (Semaphore) 的主要区别是什么？
   答案：
   - **目的**：互斥锁用于**互斥**（同一时间只允许一个访问者）；信号量用于**同步**（控制多个访问者的数量）。
   - **计数**：互斥锁本质是二值（0/1）；信号量是非负整数计数器。
   - **操作者**：互斥锁的加锁和解锁必须由**同一线程**完成；信号量的 `wait` 和 `post` 可由**不同线程**完成。

4. **题目**：在单核CPU上编写多线程程序，是否还需要考虑加锁？为什么？
   答案：**需要**。虽然单核CPU任一时刻只有一个线程在执行，但操作系统通过时间片轮转进行调度。线程可能在任何指令执行后被切换，如果多个线程访问共享数据且未加锁，会因为执行顺序不确定而产生**竞态条件**，导致结果错误。

5. **题目**：什么是死锁？产生死锁的必要条件有哪些？
   答案：死锁指两个或两个以上的线程在执行中，因争夺资源而造成的一种**互相等待**的现象。必要条件（四个）：
   i. **互斥条件**
   ii. **请求与保持条件**
   iii. **不可剥夺条件**
   iv. **循环等待条件**
6. **题目**：如何避免死锁？（说出至少一种策略）
   答案：**顺序加锁**。规定所有线程必须按照**相同的全局顺序**申请多个锁，从而破坏“循环等待”条件。

内容由AI生成仅供参考